Candidate Assignment

ShopFloor Copilot — AI-Assisted Visual Inspection & SOP Agent

Forward-Deployed Engineer (FDE) Track — Manufacturing Vertical

This exercise evaluates your practical skills as a Forward-Deployed Engineer building AI-augmented software directly against a manufacturing client's problem: Python API development, Computer Vision/OCR, Retrieval-Augmented Generation (RAG), Agentic reasoning, and adaptive optimization (online RL) — plus the judgment to know which of these to actually reach for. We estimate 4–5 hours of focused effort, to be completed and submitted within a 24-hour window from receipt.

Scenario & Objective

You are the FDE embedded with a mid-size precision-parts manufacturer (CNC machining / plastics injection). Line supervisors currently do visual QA by eye, look up SOPs and escalation rules from binders or a shared drive, and record defects on paper travelers that get keyed into the ERP hours later.

Leadership wants a single assistant — ShopFloor Copilot — that a line worker can query with a photo of a part and/or a text question (e.g. "Is this within tolerance?" or "Who do I escalate a crack to?"), and that gets measurably better at picking the right underlying tool/model configuration as it collects feedback.

Because open-source vision pipelines, OCR engines, and local LLMs differ in latency, cost, and accuracy, the system must learn — via online reinforcement learning — which configuration to route to under real feedback and latency pressure. And because this is a live shop floor, every AI-influenced decision needs an audit trail and a human checkpoint before anything irreversible happens.

Core Requirements

A. The Python API (FastAPI recommended)

- **POST /inspect:** multipart/form-data or base64 — a text query, an optional part image, and an optional work_order_id. Returns: the final answer, the defect category (if an image was supplied), any OCR-extracted work-order/part text, the pipeline configuration used (which OCR engine, which LLM, or RAG top-K), the agent's decision trace (which tools it called, in what order, and why), the execution latency, and a unique transaction ID.
- **POST /feedback:** a transaction ID plus a binary feedback score (1 for helpful, 0 for unhelpful). This must directly update the online-learning loop.
- **GET /transactions/{id}:** replays a past decision trace and the configuration used, for audit purposes.

B. Computer Vision & OCR Layer (open-source only)

- Build a lightweight defect classifier using classical OpenCV techniques (edge/contour analysis, thresholding, blob detection) and/or a small pretrained model (e.g. a YOLOv8n weight or a Hugging Face vision pipeline). Classify into 3–4 categories, e.g. no_defect / surface_scratch / crack / dimensional_deviation.
- Extract the work-order or part-ID text stamped or printed on the part using pytesseract or EasyOCR.
- Generate or source a handful of mock/synthetic part images (varying defect type, with a visible ID) — no real customer data needed.
- No managed cloud vision APIs (Rekognition, Textract, Google Vision, etc.) — the point is demonstrating the underlying CV/OCR technique, not a managed API call.

C. Retrieval-Augmented Generation (Knowledge Layer)

- Write 3–5 short mock knowledge documents: tolerance specs per defect category, rework-vs-scrap decision rules, and escalation contacts/rules.
- Implement a vector store (ChromaDB, FAISS, or a basic in-memory cosine index) to chunk and retrieve this text.

- Integrate with at least one open-source LLM via Ollama (e.g. Mistral, Llama 3, or Qwen) for grounded generation. A second model or configuration is a bonus, not a requirement — this is deliberately lighter than a 48-hour version of this exercise, given the 24-hour window.

D. Agentic Reasoning Layer

- Implement a ReAct-style loop in which the LLM decides, per query, whether to: answer directly from RAG, invoke the CV/OCR tool, invoke a mock tool check_work_order_status(work_order_id), or flag the case for human escalation.
- The decision trace returned by /inspect must show this reasoning path explicitly (which tool, in what order, and why) — on a shop floor, "the model just decided" is not an acceptable audit answer.

E. Adaptive Optimization Loop (Reinforcement Learning)

- Implement one lightweight online-learning loop — Epsilon-Greedy Contextual Bandit or simple Q-learning. Do not train a deep neural network for this.
- **State space:** the defect category detected for the query (or "no image supplied").
- **Action space:** pick ONE axis to optimize — either which OCR engine to use, or the RAG top-K — and justify in your rationale (Section G) why you chose that axis over the others.
- **Reward:** (feedback score × 10) − (latency in seconds). POST /feedback must update this loop directly, and the chosen action for a given state should visibly shift as evidence accumulates — show this in your README (a short log or a simple plot is enough).

F. Cost & Governance Guardrails

- Every AI-influenced decision (each /inspect call) must be written to an append-only audit log — a simple table or file is fine: timestamp, transaction ID, inputs, tools called, model/config used, final answer, latency.
- Any agent path that ends in "escalate" or "flag for scrap" must be returned as a proposed action, not an executed one — the API should never auto-post anywhere without a simulated human-approval step.
- Add one simple cost/latency lever: if the primary local model's response exceeds a latency ceiling you define, fall back to a smaller/faster model or a cached/templated response rather than retrying the same slow path.

Suggested Stack & Tools

You are free to use your preferred libraries; a recommended, predominantly open-source stack:

- **API:** FastAPI, Uvicorn, Pydantic.
- **Image Processing:** OpenCV, Pillow, EasyOCR or pytesseract, optionally Ultralytics YOLOv8n.
- **Agent/RAG Framework:** LangChain, LlamaIndex, or native Python — your call.
- **Vector Store:** ChromaDB, FAISS, or a basic in-memory index.
- **Models:** Ollama for local execution of open-source LLMs.

G. FDE Decision Rationale (written, ~1 page — not code)

This section is what separates an FDE from a pure backend engineer: judgment about which tool to reach for, not just the ability to build it. Answer briefly (3–5 sentences each) in your README or a separate DECISIONS.md:

1. Why vector RAG (vs. a vectorless/long-context read) fits this SOP corpus — and what would change your answer if the plant's SOP set were fixed at 20 pages instead of continuously growing.
2. Why an agent loop (vs. a fixed pipeline) is justified for the tool-selection step here — and name the one part of this system you would NOT make agentic, and why.
3. Why you chose classical CV, a pretrained model, or both for the defect classifier — and what would push you to fine-tune a custom model instead.
4. If this pilot needed to scale from one plant to ten, which cost-optimization lever — prompt/response caching, model tiering, batching, a smaller fine-tuned model, or quantization — would you apply first, and why that one before the others.

5. Name one governance risk specific to a shop floor (not a generic AI risk) that this design does not yet fully address, and how you'd close it in a real engagement.

Deliverables & Expectations

- **Codebase:** a clean, well-documented GitHub repository (or a zip, if you prefer).
- **Reproducibility:** a requirements.txt or equivalent dependency file.
- **Documentation:** a README.md with setup instructions, a breakdown of your RL strategy and observed behavior, and instructions to test each endpoint (curl or Postman examples are fine).
- **Decision Rationale:** the answers to Section G, in the README or a separate DECISIONS.md.

We will evaluate on: code quality and API design; whether the CV/OCR results are real and reproducible on open-source tooling (not a managed cloud call); the clarity and correctness of the agent's decision trace; whether the RL loop demonstrably shifts its choice as feedback accumulates; and — weighted equally with the code — the quality of your Decision Rationale. An FDE's judgment about what not to build is as much the job as what you do build.

Explicitly Out of Scope (to keep this realistic in 24 hours)

- No Flutter/mobile client or UI is required — a curl/Postman-testable API is sufficient. A minimal single-page test harness is a bonus, not a requirement.
- No managed cloud AI APIs (OpenAI, AWS, GCP, Azure) — everything runs on open-source/local tooling as above.
- No production-grade auth, multi-tenancy, or deployment infrastructure — a runnable local service is the bar.
- No deep-learning training runs — a pretrained/off-the-shelf model or classical CV is the ceiling for the vision layer, and a hand-rolled bandit (not a trained deep-RL agent) is the ceiling for the optimization layer.

> *Time-box reminder: this is scoped for ~4–5 focused hours inside a 24-hour window — if a section is eating more time than that, ship the simplest version that satisfies the requirement, note the shortcut in your README, and spend the saved time on Section G instead.*